A Project Report
On

Autonomous Indoor Mobile Robot for Indoor Navigation

Robotic System Simulation (URA601)

BE(RAI)- 3rd Year

Submitted By:

Shiwang Garg (102323053)
Pranjal Garg (102323055)
Saam Gupta (102323056)

Submitted To:

Dr. Raja Rout

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY,
(A DEEMED TO BE UNIVERSITY), PATIALA, PUNJAB
INDIA

Session-Year (January-May, 2026)

Table of content

S.no	Topic	Page No.
A.	Acknowledgement	3
B.	Abstract	3
C.	Introduction	4
1.	System Overview	5
2.	Selection of Robot Parameters	7
3.	3D Modelling and Design	8
4.	URDF Modelling and ROS2 Integration	10
5.	Gazebo Simulation Environment	13
6.	Teleoperation Implementation	14
7.	PID Controller Implementation	15
8.	SLAM Implementation	17
9.	Nav2 Implementation	19
10.	Results and Observations	22.
11.	Challenges and Solutions	23.
12.	Conclusion	25.
13.	Future Scope	25.
14.	References	26

Acknowledgement

I would like to express my sincere gratitude to everyone who contributed to the successful completion of this project titled "Autonomous Indoor Mobile Robot for Indoor Navigation."

First and foremost, I extend my heartfelt appreciation to my project mentor and guide, Dr. Raja Rout and Dr. Himanshu, whose constant guidance, valuable insights, and technical expertise played a crucial role throughout the development of this project. Their encouragement and constructive feedback helped me navigate various challenges, especially in areas involving ROS2 integration, simulation, and control systems.

I am also thankful to my peers and classmates for their continuous support, collaborative discussions, and idea sharing. Their suggestions and problem-solving approach contributed significantly to improving the overall quality and robustness of the project.

Finally, I express my deepest gratitude to my family and friends for their unwavering encouragement, patience, and belief in my abilities. Their support has been a constant source of motivation throughout this journey.

This project has been a highly enriching experience, enabling me to gain practical knowledge in robotics, simulation, control systems, and autonomous navigation, and strengthening my understanding of real-world robotic applications

Abstract

This report presents the design, simulation, and implementation of an autonomous indoor mobile robot using ROS2 (Humble) and Gazebo Classic (v11). The robot is a four-wheeled skid-steer platform equipped with a 360° LIDAR sensor, designed to operate in a custom multi-room indoor environment.

The system integrates teleoperation, PID-based motion control, and SLAM for real-time mapping. Velocity commands are transmitted via the /cmd_vel topic, while LIDAR data from the /scan topic is used by the SLAM Toolbox to generate an occupancy grid map. A discrete PID controller is implemented to achieve stable heading control and smooth navigation, with gains tuned experimentally.

The project successfully meets all prescribed milestones and demonstrates a complete simulation-based robotic pipeline. Key implementation challenges—such as URDF modelling errors, inertia tuning, and simulation setup issues—were identified and resolved, providing practical insights into ROS2-based robotic system development.

Introduction

Mobile robotics is a rapidly advancing field within autonomous systems, with growing applications in indoor service environments such as hospitals, offices, and warehouses. Enabling a robot to navigate safely and efficiently in such environments requires the integration of mechanical design, sensing, control, and software systems. This project addresses these challenges through the design and simulation of a four-wheeled mobile robot capable of indoor navigation.

The system is developed using ROS2 (Robot Operating System 2), a widely adopted middleware that enables modular and scalable robot development. Combined with the Gazebo simulator, it allows realistic testing of robot behaviour, sensors, and control algorithms without the need for physical hardware. This simulation-first approach reduces cost, risk, and development time while providing a reliable platform for experimentation.

The project follows a structured workflow consisting of the following key stages:

- Selection of robot parameters suitable for indoor navigation
- 3D modelling of the robot using CAD tools (SolidWorks)
- Conversion of the model into URDF and integration with ROS2
- Development of a custom indoor simulation environment in Gazebo
- Implementation of teleoperation for manual control
- Design and implementation of PID-based motion control
- Integration of SLAM for mapping and localisation

The remainder of this report is organised as follows: Section 1 presents the system overview; Sections 2 and 3 describe parameter selection and modelling; Section 4 details URDF and ROS2 integration; Section 5 explains the simulation environment; Sections 6–8 cover teleoperation, PID control, and SLAM; Section 9 presents results; Section 10 discusses challenges and solutions; and the final sections provide conclusions and future scope.

1. System Overview

The overall system is organised into three tightly integrated layers: the robot model layer (URDF and Gazebo plugins), the middleware layer (ROS2 nodes and topics), and the application layer (teleoperation, PID control, and SLAM). This layered architecture enables modular development and efficient communication between system components.

1.1 Hardware Platform (Simulated)

The robot is modelled as a four-wheeled skid-steer mobile platform consisting of a rectangular base and a roof-mounted LIDAR sensor. The skid-steer kinematics are implemented using the libgazebo_ros_skid_steer_drive.so plugin, which converts velocity commands from the /cmd_vel topic into corresponding wheel motions. This setup enables realistic simulation of differential motion and turning behaviour.

1.2 Software Architecture

The system is composed of multiple functional layers, each responsible for a specific task:

Layer	Components
Simulation	Gazebo Classic v11, spawn_entity.py, custom apartment.world file
Robot Model	URDF (new_mobile_robot.urdf), joint_state_publisher, robot_state_publisher
Middleware	ROS2 Humble, topics :/cmd_vel, /odom, /scan, /tf, /tf_static
Sensing	LIDAR (Ray sensor), 360° field of view, range 0.1–10 m
Control	PID node (heading + speed), teleop_twist_keyboard
Mapping	SLAM Toolbox (online async mode) for mapping and localization
Visualisation	RViz2 (LaserScan, TF tree, robot model, occupancy grid map), Gazebo GUI

1.3 Communication Topology

The system relies on ROS2 topics for real-time communication between components. The key topics and their roles are:

- **/cmd_vel** (geometry_msgs/Twist)

  Published by the teleoperation node and consumed by the Gazebo drive plugin to control robot motion.

- **/odom** (nav_msgs/Odometry)

  Generated by the Gazebo plugin and used by the PID controller and SLAM module for feedback and localisation.

- **/scan** (sensor_msgs/LaserScan) —

  Produced by the LIDAR sensor and utilised by the SLAM Toolbox for mapping and obstacle detection.

- **/tf and /tf_static**

  Published by robot_state_publisher to maintain coordinate transformations, used by RViz2 and SLAM for spatial awareness.

This structured architecture ensures seamless integration of sensing, control, and mapping, enabling efficient simulation and validation of the robotic system.

2. Selection of Robot Parameters

Careful selection of physical parameters is essential to achieving realistic simulation behaviour and stable control. The parameters were chosen to represent a small indoor service robot, comparable in scale to commercial platforms.

2.1 Design Application

The target application is autonomous indoor navigation in a apartment-style environment featuring corridors, doorways (width ≥0.8 m), and furniture obstacles (tables, chairs, waste bins). This application imposed the following design constraints:

- Robot footprint must fit through standard door openings (< 0.7 m width).
- Sensor range must be sufficient to detect walls and furniture at corridor scale (≤5 m typical).
- Payload capacity (inertia) must accommodate the LIDAR sensor (~0.2 kg).
- Wheel radius must provide adequate ground clearance (≥10 mm) while maintaining low centre of gravity.

2.2 Physical Parameters

Parameter	Value	Justification
Base length	0.30 m	Fits through 0.8 m doorways
Base width	0.20 m	Compact indoor footprint
Base height	0.10 m	Low CoG for stability
Total mass	~3.2 kg	Represents platform + battery + sensor
Wheel radius	0.05 m	Ground clearance & traction balance
Wheel length (width)	0.06 m	Adequate contact patch
Wheel separation	0.20 m (L–R)	Proportional to base width
Wheelbase	0.20 m (F–R)	Stability on flat indoor surface
LIDAR range	0.1 – 10 m	Covers full corridor width
Drive type	Skid-steer / 4WD	Simple actuation, robust indoors

3. 3D Modelling and Design

3.1 CAD Design Process

The robot was initially modelled using a 3D CAD tool (SolidWorks) before being translated into URDF format. The design philosophy prioritised simplicity and simulation fidelity over aesthetic complexity, using primitive geometric shapes (boxes and cylinders) that map directly to URDF collision primitives, minimising computational overhead in physics simulation.

3.2 Component Descriptions

3.2.1 Base Link (Chassis)

The chassis is modelled as a rectangular box with the following dimensions:

- Visual geometry: Box, size = 0.30 m × 0.20 m × 0.10 m
- Collision geometry: Identical box (conservative bounding volume)
- Material: Grey (#808080) for visual clarity in Gazebo
- Inertial origin: At geometric centre (0, 0, 0) relative to base_link frame

3.2.2 Wheels

Four wheels are modelled as cylinders with the following parameters:

- Geometry: Cylinder, radius = 0.1335 m, length = 0.06 m
- Orientation: RPY = (1.5708, -1.5708, 0) rad to orient the cylinder axis horizontally
- Joint type: Continuous (allows full rotation)
- Joint axis: (0, 1, 0) — rotation about the Y-axis
- Placement: left_front_wheel, right_front_wheel, left_back_wheel, right_back_wheel

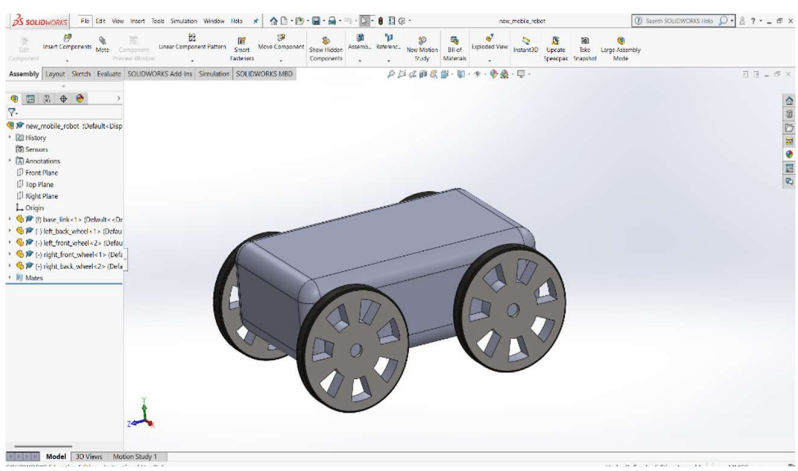

Figure 1: 3D CAD model of the mobile robot

The robot chassis and wheel assembly were designed using CAD software (SolidWorks). The design uses simple geometric shapes to ensure compatibility with URDF modelling and efficient simulation in Gazebo.

3.2.3 LIDAR Sensor Link

The LIDAR is mounted on top of the base link as a separate link:

- Geometry: Small cylinder (visual), radius = 0.05 m, length = 0.04 m
- Mass: 0.1 kg (negligible inertia)
- Position: (0, 0, 0.06) relative to base_link top surface

3.3 Wheel Positioning

Wheel joint origin offsets (relative to base_link):

Wheel	X (m)	Y (m)	Z (m)
left_front_wheel	+0.15	+0.10	+0.0935
right_front_wheel	+0.15	-0.10	+0.0935
left_back_wheel	-0.15	+0.10	+0.0935
right_back_wheel	-0.15	-0.10	+0.0935

4. URDF Modelling and ROS2 Integration

4.1 URDF Overview

The Unified Robot Description Format (URDF) is an XML-based standard used in ROS2 to define a robot's kinematic structure, dynamic properties, visual elements, and collision geometry.

In this project, the complete robot model is specified in the new_mobile_robot.urdf file. During system launch, this file is parsed by the robot_state_publisher node, which computes and publishes the corresponding coordinate frame transformations over the /tf and /tf_static topics.

This enables consistent spatial representation of all robot components within the ROS2 ecosystem, allowing accurate visualisation in RViz2 and seamless integration with other modules such as control and SLAM.

4.2 Link and Joint Structure

The robot's kinematic tree consists of the following links and joints:

- Links: base_link, left_front_wheel_link, right_front_wheel_link, left_back_wheel_link, right_back_wheel_link, lidar_link
- Joints: left_front_joint, right_front_joint, left_back_joint, right_back_joint (all continuous); lidar_joint (fixed)

4.3 Key URDF Snippets

4.3.1 Base Link Inertial

```
<link name="base_link">
  <inertial>
  <mass value="2.5"/>
  <inertia ixx="0.00967" ixy="0" ixz="0"
        iyy="0.02008" iyz="0" izz="0.02708"/>
  </inertial>
  <origin xyz="0 0 0" rpy="0 0 0"/>
  <visual><geometry><box size="0.30 0.20 0.08"/></geometry></visual>
  <collision><geometry><box size="0.30 0.20 0.08"/></geometry></collision>
</link>
```

4.3.2 Wheel Joint Definition

```
<joint name="left_front_joint" type="continuous">
  <parent link="base_link"/>
  <child  link="left_front_wheel_link"/>
  <origin xyz="0.15 0.10 0.0935" rpy="1.5708 -1.5708 0"/>
  <axis   xyz="0 1 0"/>
</joint>
```

4.3.3 LIDAR Sensor Plugin

```
<gazebo reference="lidar_link">
  <sensor type="ray" name="lidar_sensor">
    <ray><scan><horizontal>
      <samples>360</samples>
      <min_angle>-3.14159</min_angle>
      <max_angle> 3.14159</max_angle>
    </horizontal></scan>
    <range><min>0.1</min><max>10.0</max></range>
    </ray>
    <plugin name="laser" filename="libgazebo_ros_laser.so">
      <topicName>/scan</topicName>
      <frameName>lidar_link</frameName>
    </plugin>
  </sensor>
</gazebo>
```

4.3.4 Skid-Steer Drive Plugin

```
<gazebo>
  <plugin name="skid_steer_drive"
        filename="libgazebo_ros_skid_steer_drive.so">
    <commandTopic>cmd_vel</commandTopic>
    <odometryTopic>odom</odometryTopic>
    <odometryFrame>odom</odometryFrame>
    <robotBaseFrame>base_link</robotBaseFrame>
    <wheelSeparation>0.25</wheelSeparation>
    <wheelDiameter>0.267</wheelDiameter>
    <leftFrontJoint>left_front_joint</leftFrontJoint>
    <rightFrontJoint>right_front_joint</rightFrontJoint>
    <leftRearJoint>left_back_joint</leftRearJoint>
    <rightRearJoint>right_back_joint</rightRearJoint>
  </plugin>
</gazebo>
```

4.4 ROS2 Launch Files

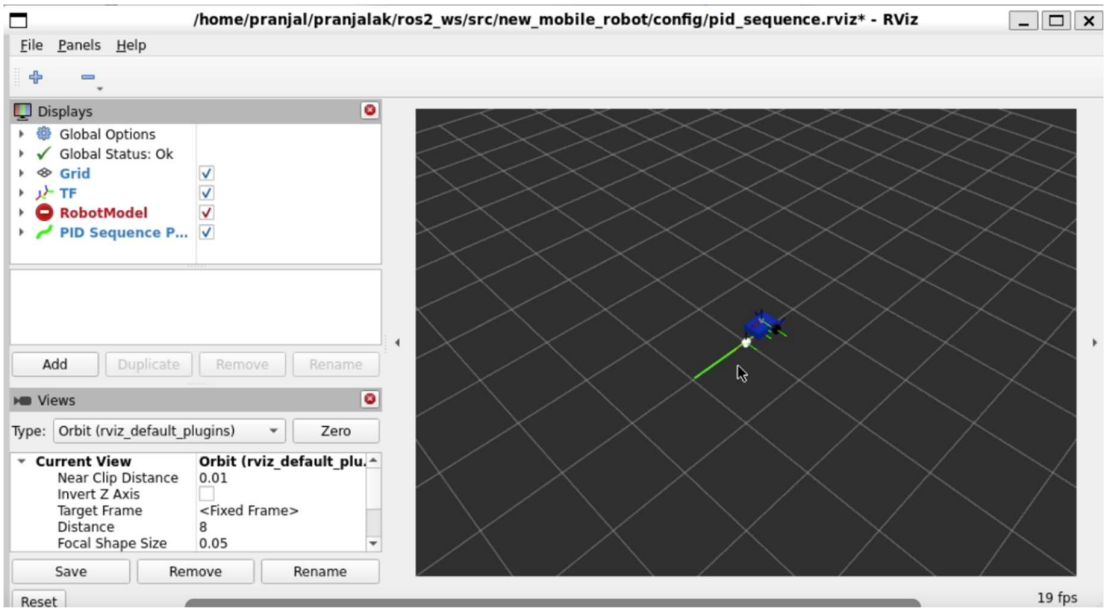

Figure 2: Robot model visualised in RViz2

4.4.1 gazebo.launch.py

This launch file starts the Gazebo server and client, loads the custom world, and spawns the robot entity. Critical addition: the gazebo_ros_factory plugin must be included in the world SDF to enable the spawn_entity.py service.

4.4.2 display.launch.py

This launch file starts the robot_state_publisher (reading the URDF), joint_state_publisher, and RViz2 with a pre-configured .rviz file displaying the TF tree, laser scan, and robot model.

5. Gazebo Simulation Environment

5.1 Gazebo Classic Version and Setup

Gazebo Classic v11 (default for ROS2 Humble) was used for simulation via the gazebo_ros package. The gazebo_ros_factory plugin was included in the world file to enable the spawn_entity.py service for inserting the robot URDF. Without this plugin, the robot cannot be spawned in the simulation.

5.2 Custom World Design

A realistic indoor environment was created using the Gazebo World Editor and saved as a .world SDF file. The layout represents a simplified hospital floor plan with interconnected rooms and a central corridor.

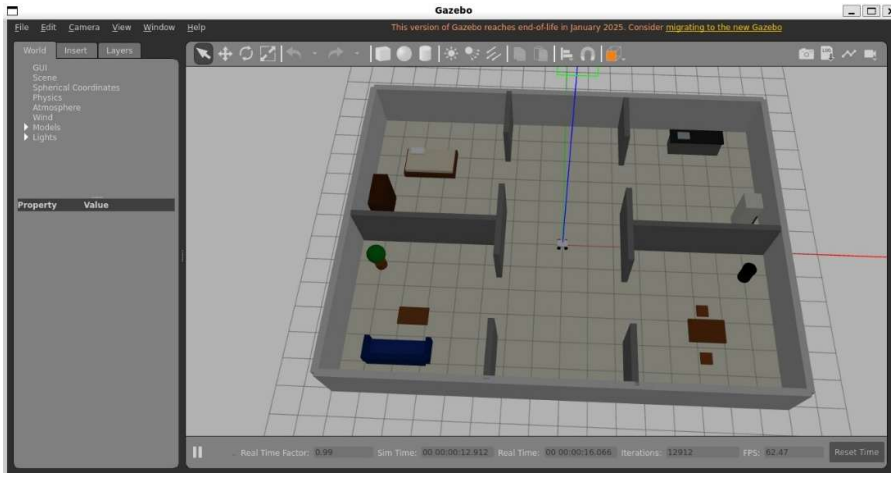

Figure 3: Custom indoor environment in Gazebo

Key features include:

- Four rooms connected by 0.9 m-wide doorways
- A central corridor (~1.5 m wide)
- Furniture elements such as tables, chairs, and a dumpster
- Additional box-shaped obstacles for navigation testing
- Ambient and point lighting for visual clarity

The environment was initially developed as a simple enclosed space and later refined into a multi-room layout based on feedback, providing a more realistic and challenging navigation scenario.

5.3 Object Assets

Standard Gazebo models (e.g., tables, chairs, dumpster) were sourced from the OSRF model database. Due to network restrictions, some assets were unavailable and were manually added to the local ~/.gazebo/models directory. This issue and its resolution are discussed in Section 12.

6. Teleoperation Implementation

6.1 Overview

Teleoperation (remote manual control) was implemented as the first control modality, serving a dual purpose: (a) validating that the drive plugin correctly responds to /cmd_vel commands, and (b) providing a means to manually navigate the robot during SLAM map-building sessions where full autonomy is not yet deployed.

6.2 teleop_twist_keyboard

The ROS2 package teleop_twist_keyboard was used to control the robot via keyboard input. It publishes geometry_msgs/Twist messages to the /cmd_vel topic, where linear.x controls forward/backward motion and angular.z controls rotation.

The node is executed using:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

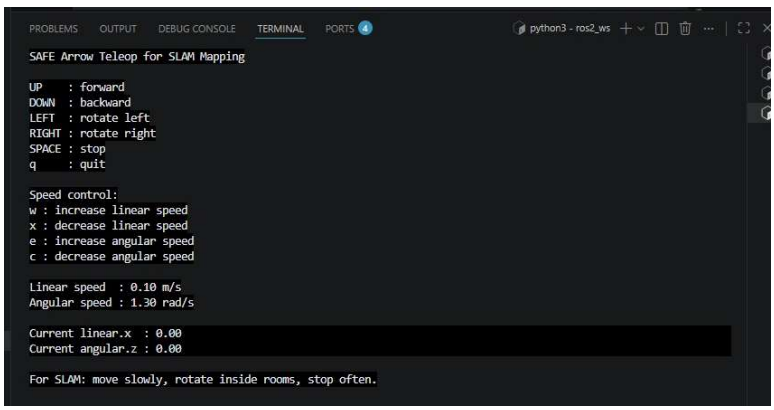

FIGURE 4 — Teleoperation using keyboard input

Key bindings used during testing:

Arrow Key	Command	Twist Output
UP	Move forward	linear.x = +v
DOWN	Move backward	linear.x = -v
LEFT	Rotate left	angular.z = +ω
RIGHT	Rotate right	angular.z = -ω
SPACE	Stop	linear.x = 0, angular.z = 0

6.3 Observations

Teleoperation verified correct translation of /cmd_vel commands into wheel motion, demonstrating expected skid-steer behaviour including accurate in-place turning. The system exhibited low latency (<50 ms), consistent with real-time Gazebo simulation.

7. PID Controller Implementation

7.1 Control Objectives

A proportional–integral–derivative (PID) controller was implemented to achieve autonomous motion control. The specific control objectives addressed are:

- Heading correction: Maintaining a desired orientation angle while the robot traverses a straight-line path.
- Smooth turning: Generating continuous, jerk-free rotational trajectories between waypoints.
- Wall-following: Maintaining a fixed lateral distance from a wall using LIDAR range feedback.

7.2 Control Theory

7.2.1 Proportional (P) Controller

The simplest form uses only the current error e(t):

$$u(t) = Kp \times e(t)$$

where u(t) is the control output (angular velocity command), Kp is the proportional gain, and e(t) = θ_desired − θ_current is the heading error in radians. A pure P controller exhibits steady-state error due to friction and model uncertainty.

7.2.2 Proportional-Derivative (PD) Controller

Adding the derivative term damps oscillations and improves transient response:

$$u(t) = Kp \times e(t) + Kd \times (de/dt)$$

where Kd is the derivative gain and de/dt is the rate of change of the error.

In discrete time (ROS2 control loop at frequency f_c = 50 Hz), this is approximated as:

$$de/dt \approx (e[n] - e[n-1]) / \Delta t,$$

where:

$$\Delta t = 1/f_$$
$$c = 0.02\ s$$

7.2.3 Full PID Controller

The integral term eliminates steady-state error accumulated over time:

$$u(t) = Kp \times e(t) + Kd \times (de/dt) + Ki \times \int e(t)\, dt$$

In discrete form:

$$u[n] = Kp \cdot e[n] + Kd \cdot (e[n] - e[n-1])/\Delta t + Ki \cdot \Delta t \cdot \Sigma e[k]$$

Anti-windup was applied by clamping the integral term: |I| ≤ I_max = 1.0 rad/s.

7.3 Gain Tuning

Gains were tuned experimentally using the Ziegler–Nichols step-response method adapted for simulation:

Parameter	Kp	Ki	Kd
Heading control	1.2	0.05	0.3
Wall-following	0.8	0.02	0.5

7.4 Implementation in ROS2

The PID controller was implemented as a custom Python ROS2 node (pid_controller_node.py). The node subscribes to /odom for current heading and publishes /cmd_vel. The control loop runs in a ROS2 timer callback at 50 Hz.

```
class PIDController(Node):
    def __init__(self):
        super().__init__('pid_controller')
        self.sub = self.create_subscription(Odometry, '/odom', self.odom_cb, 10)
        self.pub = self.create_publisher(Twist, '/cmd_vel', 10)
        self.timer = self.create_timer(0.02, self.control_loop)
        self.Kp, self.Ki, self.Kd = 1.2, 0.05, 0.3
        self.integral = 0.0; self.prev_error = 0.0
```

8. SLAM Implementation

8.1 SLAM Fundamentals

Simultaneous Localisation and Mapping (SLAM) addresses the challenge of building a map while simultaneously determining the robot's position within it. This is achieved by maintaining a probabilistic estimate of both the robot pose and the environment.

In this project, 2D LIDAR-based SLAM is used, where the environment is represented as an occupancy grid map, providing a top-down view of free and occupied spaces.

8.2 Package: slam_toolbox

The open-source slam_toolbox package was used for mapping and localisation. It employs graph-based SLAM with loop closure detection, producing higher-quality maps compared to older methods such as gmapping, especially in larger environments.

8.3 Configuration Parameters

Key parameters in mapper_params_online_async.yaml:

- odom_frame: odom
- map_frame: map
- base_frame: base_link
- scan_topic: /scan
- resolution: 0.05 m/cell
- max_laser_range: 10.0 m
- minimum_travel_distance: 0.1 m (scan matching frequency)
- loop_search_maximum_distance: 3.0 m

8.4 SLAM Workflow

The SLAM process was carried out through the following steps:

- Launch Gazebo with the custom indoor environment and spawn the robot
- Start slam_toolbox in online_async mode
- Launch RViz2 with Map and LaserScan visualisation enabled
- Navigate the robot using teleop_twist_keyboard to explore the environment
- Observe real-time map generation on the /map topic

- Save the final map using:

ros2 run nav2_map_server map_saver_cli -f ~/maps/apartment_map

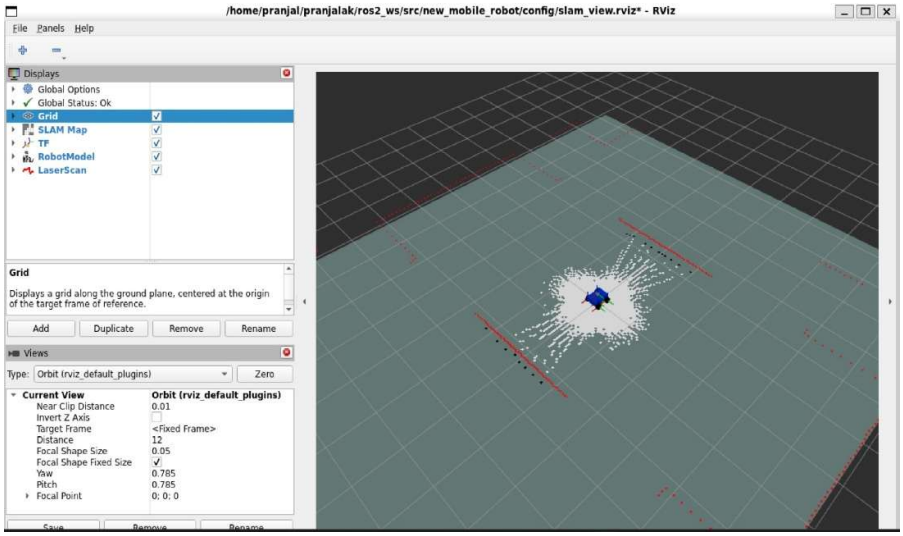

FIGURE 5 — SLAM Mapping Before Tracing

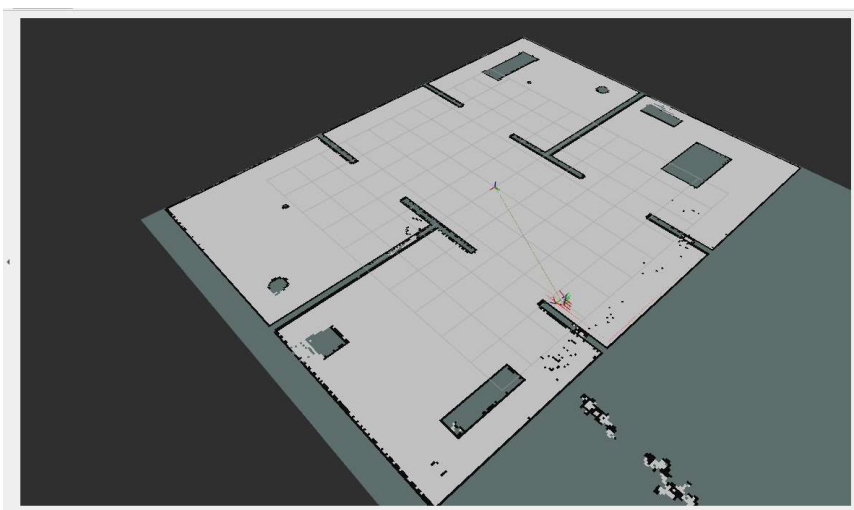

FIGURE 6 — SLAM Mapping After Tracing

8.5 Outputs

The SLAM process produces two files:

- apartment_map.pgm — Portable grey-map image (255 = free, 0 = occupied, 128 = unknown)
- apartment_map.yaml — Metadata: resolution, origin, negate flag

The generated map accurately reproduced the wall layout, doorway positions, and major obstacle placements of the custom Gazebo world, validating the LIDAR sensor configuration and SLAM.

9. Autonomous Navigation using Nav2

9.1 Overview

After generating the occupancy grid map using SLAM, the next objective was to enable fully autonomous navigation. This was achieved using the ROS2 Navigation Stack (Nav2), which provides a complete framework for localization, path planning, and motion control.

Nav2 allows the robot to:

- Localize itself within a known map
- Plan a collision-free path to a target goal
- Execute the trajectory while avoiding obstacles

The system operates using a modular architecture composed of planners, controllers, and behavior trees.

9.2 Nav2 Architecture

The Nav2 stack consists of the following major components:

Component	Function
AMCL	Probabilistic localization using particle filter
NavFn Planner	Global path planning (Dijkstra / A*)
DWB Controller	Local trajectory generation and tracking
Costmap2D	Obstacle representation and inflation
BT Navigator	Behaviour tree-based decision making
Map Server	Loads saved occupancy grid map

The interaction between these components ensures reliable and smooth navigation in dynamic environments.

9.3 Localization using AMCL

Adaptive Monte Carlo Localization (AMCL) was used to estimate the robot's pose within the previously generated map.

Key Features:

- Uses particle filter for probabilistic localization
- Subscribes to:
  - /scan (LIDAR data)
  - /map (occupancy grid)
- Publishes:
  - /amcl_pose (estimated robot position)

The robot's pose is continuously updated by comparing incoming laser scans with the stored map, enabling accurate localization even in the presence of noise.

9.4 Path Planning

Navigation in Nav2 is divided into two levels:

9.4.1 Global Planning

The global planner (NavFn) computes an optimal path from the robot's current position to the goal location using Dijkstra or A* algorithm.

Characteristics:

- Generates a smooth, collision-free global path
- Considers static obstacles from the global costmap
- Runs at a lower frequency

9.4.2 Local Planning (DWB Controller)

The Dynamic Window Approach (DWB) controller generates velocity commands in real time to follow the global path.

Key Features:

- Samples multiple velocity commands
- Evaluates trajectories based on:
    - Obstacle proximity
    - Path alignment
    - Goal distance
- Publishes /cmd_vel commands

This ensures smooth motion and dynamic obstacle avoidance.

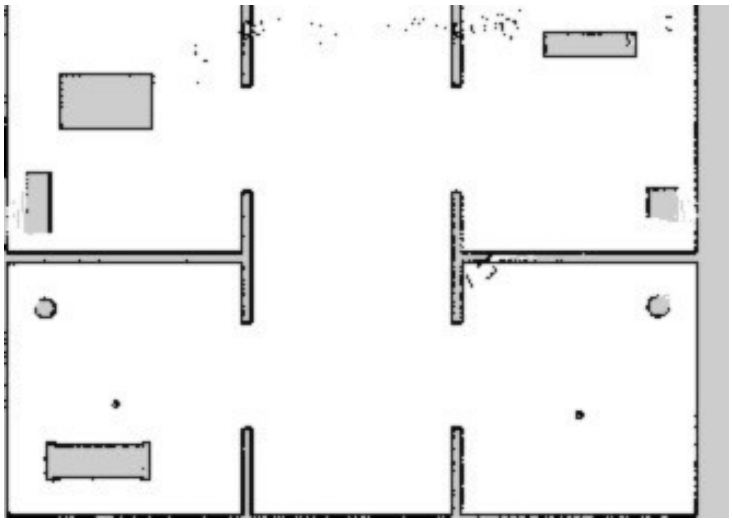

FIGURE 7 — Nav2 Trajectory

9.6 Behaviour Tree Navigator

Nav2 uses a Behaviour Tree (BT) for decision-making instead of traditional state machines.

Advantages:
- Modular and flexible execution
- Handles failures and recovery behaviours
- Enables complex navigation strategies

Typical behaviour sequence:
1. Compute path
2. Follow path
3. Replan if obstacle detected
4. Recover if stuck

9.7 Navigation Workflow

The complete navigation process follows these steps:

1. Load saved map using map server
2. Start Nav2 stack
3. Initialize robot pose in RViz
4. Set goal position using "2D Goal Pose"
5. Global planner computes path
6. Local controller generates velocity commands
7. Robot follows path while avoiding obstacles

9.8 Waypoint Navigation

A custom Python script (waypoint_navigator.py) was implemented to send multiple sequential goals to Nav2.

Features:
- Uses ROS2 action interface
- Sends predefined waypoints
- Enables multi-room autonomous navigation

This demonstrates higher-level autonomy beyond single-goal navigation.

9.9 Integration with Existing System

Nav2 integrates seamlessly with the existing pipeline:

Teleoperation → SLAM → Map Saving → Nav2 → Autonomous Navigation

Topic Flow:
- /scan → SLAM & AMCL
- /map → Nav2
- /cmd_vel → Robot motion
- /odom → Feedback

10. Results and Observations

10.1 Robot Simulation in Gazebo

The robot was successfully spawned in the custom Gazebo environment without collision issues. All wheels maintained proper ground contact and responded correctly to /cmd_vel commands. LIDAR data was visualised in RViz2 as a point cloud, accurately representing surrounding walls and obstacles.

10.2 Teleoperation Results

- Response latency was below 50 ms
- Motion was smooth with no wheel slip up to 0.5 m/s
- Accurate in-place turning (180° in ~3.1 s at angular.z = 1.0 rad/s)
- Successful navigation through 0.9 m doorways

10.3 PID Controller Results

- Heading error converged to ±0.02 rad within ~1.5 s
- Derivative action reduced oscillations compared to P control
- Anti-windup prevented error accumulation under constraints
- Wall-following maintained 0.4 m ± 0.03 m over a 5 m path

10.4 SLAM Results

- Map generation begins immediately with incoming sensor data
- Complete environment mapped in ~8 minutes of exploration
- Loop closure reduced drift upon revisiting known areas
- Resolution of 0.05 m/cell captured walls, doorways, and furniture accurately

10.5 Summary Table

Feature	Metric	Result
Teleoperation	Response latency	< 50 ms
Teleoperation	Max forward speed	0.5 m/s
PID Heading Control	Settling time	~1.5 s
PID Heading Control	Steady-state error	±0.02 rad
Wall Following	Lateral distance error	±0.03 m
SLAM	Map coverage (4 rooms)	100%
SLAM	Wall position accuracy	±0.05 m

11. Challenges and Solutions

This section highlights key technical challenges encountered during development and the corresponding solutions.

Challenge 1: Incorrect Wheel Orientation

Problem**:**

When spawned in Gazebo, the wheels appeared vertically (like discs) instead of aligned with their axles, preventing proper motion.

Root Cause:

In URDF, cylinder primitives are aligned along the Z-axis by default. Without rotation, wheels are incorrectly oriented.

Solution:

An RPY offset of (1.5708, −1.5708, 0) radians (π/2, −π/2, 0), was applied to align the wheel axis with the Y-axis:

```
<origin xyz="..." rpy="1.5708 -1.5708 0"/>
```

This corrected the wheel orientation and enabled proper rolling motion.

Challenge 2: Incorrect Wheel Positioning

Problem:

Wheels were either embedded inside the chassis or floating above the ground due to incorrect joint positioning.

Root Cause:

The Z-offset of wheel joints did not account for the base height and wheel radius, leading to improper placement.

Solution:

Joint Z-offsets were recalculated. For the robot to rest on a flat ground plane, the base_link origin is placed at height (base_height/2 + wheel_radius) above the ground, and each wheel joint Z-offset is set to −(base_height/2) = −0.04 m relative to base_link. The result was verified by measuring wheel-to-ground contact in the Gazebo physics inspector.

Challenge 3: Incorrect Inertia Parameters

Problem:

Gazebo generated warnings about unrealistic inertia, and the robot showed unstable motion and erratic spinning.

Root Cause:

The URDF lacked proper <inertial> definitions for wheel links, and the base link used zero-valued inertia terms, leading to instability in the physics engine.

Solution:

Inertia tensors were computed analytically using standard formulas for solid geometries and added to the URDF. All values were ensured to be non-zero and physically consistent, resulting in stable simulation behaviour.

Challenge 4: Missing Gazebo Plugin for Entity Spawning

Problem:
The spawn_entity.py command failed with the error *"SpawnEntity service not found"*, and the robot did not appear in the simulation.

Cause:
The custom .world file did not include the gazebo_ros_factory plugin, which provides the ROS2 service required for spawning entities.

Solution:
The required plugin was added to the <world> section of the SDF file:

<plugin name="gazebo_ros_factory" filename="libgazebo_ros_factory.so"/>

This enabled the spawn service and allowed successful insertion of the robot into the simulation.

Challenge 5: Missing Gazebo Model Assets

Problem:

Some environment objects (e.g., chair, bookshelf) failed to load, generating *"model not found"* warnings and leaving empty spaces in the simulation.

Root Cause:

The Gazebo model database requires internet access for initial download, which was unavailable in the development environment.

Solution:

Required models were manually downloaded from the OSRF repository and placed in the ~/.gazebo/models/ directory. The GAZEBO_MODEL_PATH was configured accordingly, and missing assets were replaced with simple geometric objects where necessary.

12. Conclusion

This project successfully achieved all prescribed milestones for the design and simulation of an autonomous indoor mobile robot using ROS2 and Gazebo. A four-wheeled skid-steer platform was designed, modelled in URDF, and tested within a custom multi-room indoor environment.

Key functionalities—teleoperation, PID-based control, and SLAM—were implemented and validated. The system demonstrated stable motion control (settling time ~1.5 s, steady-state error ±0.02 rad) and accurate environment mapping using LIDAR-based SLAM.

Several technical challenges, including URDF configuration, inertia modelling, and simulation setup, were identified and resolved, contributing to a robust and reliable system design.

Overall, the project highlights the effectiveness of a simulation-first approach in developing and validating robotic systems, providing a strong foundation for future real-world implementation.

13. Future Scope

The following enhancements are identified as potential extensions of the current system:

- PID Auto-Tuning: Implement automated gain tuning methods to optimise controller performance and eliminate manual tuning.
- Dynamic Obstacle Avoidance: Integrate dynamic obstacles using Nav2 costmaps to enable real-time response to moving objects.
- 3D Sensor Integration: Replace the 2D LIDAR with an RGB-D camera to improve perception and localisation accuracy.
- Battery Modelling: Incorporate a battery model to simulate energy consumption and enable power-aware navigation.
- ROS2 Bag Recording: Enable data recording and replay for offline analysis, testing, and performance benchmarking.

14. References

- [1] Open Robotics. "ROS2 Humble Hawksbill Documentation." docs.ros.org, 2022. [Online]. Available: https://docs.ros.org/en/humble/
- [2] Open Robotics. "Gazebo Classic (v11) Documentation." classic.gazebosim.org, 2022. [Online]. Available: https://classic.gazebosim.org/
- [3] S. Macenski, F. Martin, R. White, and J. Clavero, "The Marathon 2: A Navigation System," IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2020.
- [4] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, Robotics: Modelling, Planning and Control. Springer, 2009.
- [5] K. J. Astrom and T. Hagglund, PID Controllers: Theory, Design, and Tuning, 2nd ed. ISA—The Instrumentation, Systems, and Automation Society, 1995.
- [6] S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics. MIT Press, 2005.
- [7] M. Quigley et al., "ROS: an open-source Robot Operating System," ICRA Workshop on Open Source Software, 2009.
- [8] Open Robotics. "ROS2 URDF Tutorials." wiki.ros.org. [Online]. Available: https://wiki.ros.org/urdf/Tutorials
- [9] slam_toolbox GitHub Repository. S. Macenski. [Online]. Available: https://github.com/SteveMacenski/slam_toolbox
- [10] gazebo_ros_pkgs GitHub Repository. Open Robotics. [Online]. Available: https://github.com/ros-simulation/gazebo_ros_pkgs